

**PES University, Bengaluru**

(Established under Karnataka Act 16 of 2013)

END SEMESTER ASSESSMENT (ESA) - May 2023**UE19CS351 - Compiler Design****Total Marks : 100.0**

1.a. With a diagram explain the various phases of a compiler. You must clearly depict every input and output to each phase. (8.0 Marks)

1.b. Why is buffering used in lexical analysis? What are the commonly used buffering methods? (4.0 Marks)

1.c. What are Lexical Errors? Write down the sequence of lexemes for the following code segment
`while(i <= 10) { printf("%d",i); i++; }` (4.0 Marks)

1.d. Construct transition diagrams for whitespaces and identifiers. (4.0 Marks)

2.a. Explain in brief any two Error-Recovery Strategies. (4.0 Marks)

2.b. Consider the following grammar: $S \rightarrow +SS \mid *SS \mid a$

1. Calculate FIRST and FOLLOW
2. Construct predictive parsing table
3. Show how to parse the input $+aa$

(6.0 Marks)

2.c. Consider the following grammar: $S \rightarrow SS+ \mid SS^* \mid a$

1. Construct LR(0) automata
2. Construct SLR parsing table
3. Show how to parse the input $aa+$ using SLR parser

(10.0 Marks)

3.a. Write short notes on:
1. Types of attributes
2. Types of Syntax Directed Translations (SDTs)

(8.0 Marks)

3.b. Consider the following grammar:
 $D \rightarrow TL$
 $T \rightarrow \text{int} \mid \text{float}$
 $L \rightarrow L_1, \text{id} \mid \text{id}$
Write semantic rules to add/update type(int/float) information of the variables in declaration statement to the symbol table during parsing.

(4.0 Marks)

3.c. Write syntax directed definition (SDD) to generate intermediate code for for-loop.
Given: $S \rightarrow \text{for}(S_1; C; S_3)S_4$

(4.0 Marks)



3.d. Write syntax directed definition (SDD) to generate three-address intermediate code for given Conditional Boolean expression production.
 $C \rightarrow B_1 \mid B_2$
Note: Use function *new()* that generates new labels.

(4.0 Marks)

4.a. Write short notes on:

1. Quadruples
2. Static Single Assignment (SSA)

(5.0 Marks)

4.b. Construct control-flow graph (CFG) for the following three-address code.

(5.0 Marks)

```
s = 0
a = 4
i = 0
ifFalse k == 0 goto L1
b = 1
goto L2
L1 : b = 2
L2 : ifFalse i < n goto L3
    t1 = a * b
    t2 = s + t1
    s = t2
    t3 = i + 1
    i = t3
    goto L2
L3 : return s
```

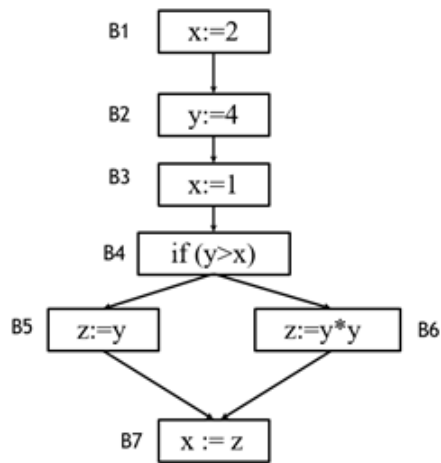
4.c. Optimize the code below by applying the following code transformations: Constant Folding, Constant Propagation and dead code elimination.

(5.0 Marks)

```
int a = 30;
int b = 9 - (a / 5);
int c;
c = b * 4;

if (c > 10) {
    c = c - 10;
}
return c * (60 / a);
```

4.d. Calculate use[B], def[B] and then find IN[B], OUT[B] by applying *Live-variable Analysis Algorithm* on the following flow graph.



(5.0 Marks)

5.a. Explain in brief any four issues in the design of a code generator. (4.0 Marks)

5.b. Explain the following with a neat diagram

1. Activation record
2. Activation tree

(6.0 Marks)

5.c. Explain in brief the different ways in which a procedure can access nonlocal data on a runtime Stack. (4.0 Marks)

5.d. Generate target code sequence for the following basic block three address statements with register and address descriptors.

1. $x := y + z$
2. $z := x * x$
3. $y := z$
4. $x := y + z$

Note 1: Assuming only two registers (R1 and R2) are available and all the variables are live on exit from the basic block. No temporary variables are used.

Note 2: Assuming that all arithmetic operations take their operands from registers. (6.0 Marks)